Architecture Principles Established

This decision establishes three principles that govern the platform design going forward:

Principle 1 — Configurability has a hard boundary

The platform is configurable up to the boundary of federal law. Anything mandated by federal regulation (CFR, USC, USDA policy) is implemented in code and protected by IsFederal guardrails. It cannot be exposed as a configuration surface regardless of how the request is framed. This boundary must be explicit in every component design.

Principle 2 — Vertical depth over horizontal breadth

The platform is purpose-built for CNP program administration. Design decisions optimize for federal nutrition program workflows, not for general applicability to other program types or industries. When a design choice trades off between "works for any program" and "works perfectly for SNP/CACFP/SFSP," the latter wins.

Principle 3 — Framing must match audience

Technical and executive audiences require different language for the same system. Using BPM + ERP with engineers produces compliance risk. Using "configurable compliance engine" with state agency CIOs produces confusion. Both framings are valid; both have designated contexts. No document should attempt to serve both audiences simultaneously.

Alternatives Considered

Alternative	Verdict	Reason
Keep BPM + ERP in all documents	Rejected	Creates compliance risk when used in engineering contexts. An engineer building a true BPM engine to spec would produce a system with no federal guardrails.
Remove BPM + ERP from all documents including executive comms	Rejected	The framing has real value for executive alignment and nTIG state conversations. Removing it entirely loses useful communication leverage without engineering benefit.

Add a disclaimer to the BPM + ERP framing rather than replacing it	Rejected	Disclaimers are not read. A spec that says "BPM + ERP (but not really BPM, and not really ERP)" will be interpreted as BPM + ERP by anyone who skims it.
Use "Rules Engine + Financial System"	Rejected	Accurate but incomplete. Misses the workflow management, compliance administration, and multi-tenant dimensions. Also carries its own technical framing baggage (Drools, Blaze Advisor).
Use "Configurable Compliance Engine" in engineering docs, BPM + ERP in executive docs	Accepted	Each framing is used in the context where it serves its audience correctly. No single document attempts to serve both audiences.

Name this system a "configurable compliance platform," not a BPM+ERP

Context

During the offsite in Denver, the platform was described as a "BPM + ERP." This framing misrepresents the actual technical design, which is a configurable federal compliance engine. This is not a workflow orchestrator on top of a financial management system.

Decision

Engineering documentation uses "configurable federal nutrition compliance platform." Executive and nTIG communications may retain "BPM + ERP" as shorthand where audience familiarity requires it. The framing used must match the audience.

Rationale

- BPM implies arbitrary process orchestration — our workflow engine is constrained to federal program compliance flows with hard federal guardrails.
- ERP implies a general-purpose financial system — our accounting layer is program-specific (federal reimbursement math), not general ledger.
- Accurate internal framing prevents engineers from making incorrect assumptions about scope and flexibility.
- "Configurable compliance platform" communicates the actual value: state-specific behavior delivered through configuration, not code forks.

Append-only audit tables instead of event sourcing

Context

The requirements specification called for "event sourcing." After review, the five stated audit requirements are:

1. Who changed what and when,
2. Point-in-time reconstruction of any record
3. Application state at approval
4. Claim state at payment
5. Form content at submission.

Decision

Use three append-only audit tables:

1. `AuditLogs` Who/what/when for every state change
2. `EntitySnapshots` Full JSONB record snapshot at key lifecycle events
3. `FieldChangeLog` Individual field old/new values

No event sourcing infrastructure.

Rationale

- Event sourcing requires event store infrastructure, projection rebuilding, eventual consistency handling, and schema evolution discipline. This is significant complexity added without additional .
- `EntitySnapshots` (JSONB) satisfies point-in-time reconstruction without event replay.
- `FieldChangeLog` satisfies "who changed what" with a simple SQL query, not a projection computation.
- Application and claim state at approval/payment are already covered by the `ClaimLaws` freeze pattern (ADR-related). `EntitySnapshots` provides the machine-queryable companion to the PDF `FormSnapshot`.
- Append-only means rows are never updated or deleted. Audit integrity is structural, not enforced by application logic.

Rejected alternatives

- Event sourcing: substantial infrastructure cost and complexity for no additional audit coverage beyond what append-only tables provide.
- Database triggers - untestable, difficult to maintain, not portable across schema changes.

Consequences

- All audit writes go through `IAuditService` — no direct table writes from application code.
- `FormSnapshot` (PDF to S3) remains — it is the human-readable legal record. `EntitySnapshots` is the machine-queryable companion. Both are required.
- Rows in audit tables are immutable. Schema changes add columns — never modify existing ones.

Named hooks for workflow conditions and actions, not expression language

Context

The workflow engine stores state machine definitions in the database (configurable per state). Workflow transitions can have conditions (must be true to allow the transition) and actions (execute when the transition fires). We needed to address how conditions and actions are represented in configuration: Arbitrary expressions evaluated at runtime or named references to compiled code.

Decision

Conditions and actions are stored as named hook identifiers (e.g., `NoNDLMatch`, `FreezeClaimLaws`, `AllClaimErrorsResolved`). The engine resolves the identifier to a compiled implementation at runtime.

Rationale

- Federal math (CEP multiplier, income eligibility thresholds, NDL check logic) must never be encoded in configuration strings. If it is encoded, we could accidentally configure it differently for a state.
- Named hooks are testable. Each hook is a C# class with unit tests. An expression language evaluated at runtime is not independently testable.
- Named hooks are auditable. The codebase is the authoritative record of what each hook does. An expression language puts business logic in the database where it can be changed without a deployment review.
- State-specific configuration is achieved through the referenced hooks, not by rewriting what hooks do.

Rejected alternatives

- Expression language (e.g., NCalc, MVEL, custom DSL): allows federal math to leak into configuration; bypasses code review; untestable in isolation.

- Stored procedure references: tight coupling to a specific database.

Consequences

- Every condition and action hook is a named, registered implementation. Adding a new hook requires a deployment.
- The configuration team cannot define their own hooks. They select from the registered catalog via the Workflow Override API.
- The `IsFederal=true` flag on any condition or action makes that hook immutable to tenant override (see ADR-007).
- Hook names must be stable. Renaming requires a migration of the `WorkflowTransitionConditions` and `WorkflowTransitionActions` tables.

Three-layer configuration hierarchy: platform → program → tenant

Context

The current CTS system has ~3,000 configurable values spread across hardcoded defaults, stored procedures, config files, and per-state database records — no consistent resolution order, no validation bounds, no effective dating. Adding a new state requires locating and replicating all relevant values with no documented process.

Decision

`ICnpConfigService.GetValue<T>(configKey, tenantCode, programCode?)` *resolves values in order: (1) active tenant override, (2) program default, (3) platform default. Every config key is defined in* `ConfigDefinitions` *(cnp_global) with a data type, validation bounds, federal lock flag, and federal citation. Overrides are stored in* `TenantConfigOverrides` *(cnp_ops) with effective dating.*

Rationale

- Platform default ensures every key has a value — no null reference possible.
- Program layer allows CACFP to have different defaults than NSLP without requiring per-state configuration for the difference.
- Tenant layer allows per-state variation within the bounds set at the platform level.
- Effective dating means rate changes (e.g., annual FNS reimbursement rate updates) are pre-staged and activate automatically — no emergency deployments on October 1.
- Validation bounds (`MinValue` , `MaxValue`) prevent misconfiguration. A state cannot set a reimbursement rate outside the federally allowed range.

- Centralizing ~3,000 values in one service eliminates the scatter that makes CTS impossible to audit.

Rejected alternatives

- appsettings.json / environment variables: requires deployment for every config change; cannot be state-specific without per-state deployments.
- Single flat config table with TenantId: no resolution hierarchy; no program-level grouping; no bounds validation.

Consequences

- All application code reads config through `ICnpConfigService` — no direct table reads for configuration values.
- Adding a new state uses `POST /tenants/{tenant}/scaffold?similarTo=UT` to copy a comparable state's overrides as a starting point. Zero-deployment onboarding.
- LaunchDarkly manages ~47 module-level feature flags (on/off per state). `CnpDefaults` manages ~3,000 fine-grained values. These are separate systems with different purposes and must not be conflated.

Database-per-tenant: separate Aurora schema per state, no TenantId column

Context

Multi-tenant SaaS platforms typically use one of three patterns: shared database with shared schema (TenantId column on every table), shared database with separate schemas, or separate database per tenant. CNP handles child eligibility data for state agencies that have independent regulatory authority and FERPA obligations. Cross-state data leakage — even accidental — would be a regulatory violation.

Decision

Each state agency gets its own Aurora schema within `cnp_ops`*. The schema name is the tenant boundary (e.g.,* `cnp_ops.ut`*,* `cnp_ops.wi`*). No* `TenantId` *column exists in Aurora. Cross-state data access is structurally impossible — not just policy-controlled. A* `TenantCode` *column is added by the CDC pipeline during replication to Snowflake only, where all states' data is aggregated for analytics.*

Rationale

- A shared schema with TenantId requires row-level security filters on every query. A missing WHERE clause exposes every state's data. The failure mode is silent and catastrophic.

- Schema-per-tenant makes cross-state access impossible at the database level. No query can return another state's data regardless of application bugs.
- State agencies have independent data governance requirements. Schema isolation provides a clean audit boundary — a state's DBA can confirm their data is isolated without trusting application-level filtering.
- Schema-per-tenant allows independent schema migration per state (important during phased rollout — different states on different platform versions simultaneously).
- Snowflake adds TenantCode at replication because the analytics warehouse aggregates all states by design and requires an explicit discriminator. This is correct and not inconsistent with Aurora's isolation model.

Rejected alternatives

- Shared schema with TenantId — one missing WHERE clause exposes all states' child data; RLS policies are difficult to audit; fails regulatory bar for state agency data isolation.
- Separate RDS cluster per state — cost-prohibitive at 28 states; schema migrations require 28 independent runs; connection pool overhead.

Consequences

- Connection strings are per-schema. The application selects the schema based on the authenticated tenant context from the JWT — never from user input.
- Schema migrations run per state on the deployment schedule for that state. A migration that breaks one state does not affect others.
- cnp_global (387 shared reference tables) uses a single schema shared across all states — this is read-only reference data, not operational data, and requires no isolation.
- SFSP seasonal deploy blackout (May–August) applies per state — schema isolation makes it possible to hold a state's schema at a prior version during their operational blackout.

RBAC permission catalog instead of CnpObjects ACL

Context

The current system manages authorization through `CnpObjects` — an ACL model with 5,082 permission rows, including approximately 1,500 rows for report-level access. Every object in the system has its own ACL entry. Adding a new object requires a new row. Auditing who can do what requires joining across multiple tables and is rarely legible to non-engineers.

Decision

Replace the 5,082-row ACL with a role-based catalog of ~180 `PermissionCatalog` entries organized by domain (SECURITY, CONFIGURATION, APPLICATIONS, CLAIMS, etc.). Nine system roles. Role-permission assignments in `RolePermissions`. Per-state overrides in `TenantRoleOverrides`. Access check: `ICnpPermissionService.HasPermission(userId, tenantCode, permissionCode)`.

Rationale

- ~1,500 report-specific ACL rows collapse into domain-level permissions (e.g., `REPORTING.FINANCIAL.READ`). Report access is governed by the domain the report covers — adding a new report does not require a new permission row.
- Roles are legible to state agency administrators. "Caseworker" vs. "Reviewer" vs. "SponsorDirector" communicates the intent. ACL entries do not.
- `ScopeType` on `UserRoleAssignments` (AllSponsors | SpecificSponsor | OwnDataOnly) replaces per-object ACL rows for sponsor-scoped access — the most common per-state variation.
- Nine system roles cover the full user population across all states. Per-state role customization is handled through `TenantRoleOverrides`, not a new permission model.
- Auditing "who has access to what" is now a single query against `PermissionCatalog` + `RolePermissions` + `UserRoleAssignments` — legible to a compliance auditor without engineering assistance.

Rejected alternatives

- Retain CnpObjects model — 5,082 rows is not maintainable; adding modules multiplies the row count; FNS auditors cannot interpret it independently.
- Attribute-based access control (ABAC) — more expressive than needed; runtime policy evaluation is slower and harder to audit; overkill for nine roles.

Consequences

- Migration path: SECURITY domain permissions (68 entries) go live first. CONFIGURATION domain (347 entries) second. Full migration is not required before Phase 1 launch — CnpObjects can coexist during transition.
- The JWT carries roles and tenant scopes. It does not carry the full permission list — permissions are resolved server-side by `ICnpPermissionService`.
- IsFederal on `PermissionCatalog` entries blocks state override of federally required access controls (e.g., FNSAuditor read access to all records).

ADR-007

IsFederal guardrail: structural block on federally mandated values

Context

The platform is designed to be configurable per state. However, federal law mandates specific calculations, rates, and behaviors that states cannot legally modify — the CEP claiming percentage multiplier (1.6×), income eligibility thresholds, the NDL check requirement, and others. A fully configurable system without guardrails can be misconfigured to produce incorrect federal reimbursements, which exposes both the state and LINQ to FNS audit findings and potential liability.

Decision

`IsFederal=true` *is a structural lock applied at four levels: config values (* `ConfigDefinitions` *), workflow states (* `WorkflowStates` *), workflow transition conditions and actions (* `WorkflowTransitionConditions` *,* `WorkflowTransitionActions` *), and permission catalog entries (* `PermissionCatalog` *). When* `IsFederal=true` *, no tenant override is accepted — the API returns HTTP 403 with a federal citation. No override mechanism exists. This is enforced in code, not policy.*

Rationale

- The CEP ISP multiplier (1.6×) is mandated by 42 USC 1759a. It is not a reasonable default — it is a legal requirement. Allowing a state to configure it to 1.5× would silently produce incorrect reimbursements for every CEP school in that state.
- Policy-level controls (documentation, training, UI warnings) are insufficient. The historical record shows states and implementation teams do misconfigure values when the system allows it.
- A hard code block is the only control that survives staff turnover, onboarding errors, and edge cases where an administrator believes they are making a legitimate change.
- `FederalCitation` stored alongside the lock provides the specific statutory or regulatory reference, making the block auditable and defensible to FNS.

Rejected alternatives

- Warning UI with acknowledgment — user dismisses warning; misconfiguration still occurs.
- Approval workflow for federal overrides — adds a human gate, which introduces the risk that a reviewer approves an incorrect override in good faith.

- Audit-only, no block — detects the problem after incorrect reimbursements have already been submitted to FNS.

Consequences

- The list of `IsFederal=true` values must be maintained as federal regulations change. Annual FNS rate updates that change the lock value require a platform deployment — they cannot be config-updated.
- LINQ engineering owns the list of federal locks. State agencies and LINQ configuration staff cannot modify it.
- The NDL check (`NoNDLMatch` condition hook, `IsFederal=true`) means the approval workflow cannot proceed if an NDL match exists — this is non-negotiable regardless of state configuration.

NLP analytics uses an approved template catalog, not arbitrary SQL generation

Context

The analytics layer exposes natural language querying for state agency staff (e.g., "how many applications are pending for district 5 this month"). An LLM can translate natural language to SQL. However, the analytics warehouse aggregates all states' data in Snowflake — arbitrary SQL generation creates the risk of cross-state data access and PII exposure depending on how a question is phrased or how the model hallucinates a query.

Decision

The LLM maps a natural language question to a template ID in `ApprovedQueryTemplates` *and extracts parameters (date ranges, district IDs, program codes). It does not generate SQL.* `TenantCode` *is injected from the authenticated session — never from the user's question. If no template matches, the query is rejected with a message explaining what question types are supported. Every query attempt is logged in* `QueryLog` *including a* `CrossTenantFlag`.

Rationale

- Arbitrary SQL generation over a multi-state warehouse has no reliable way to prevent a cleverly phrased question from returning another state's data.
- Approved templates are auditable: every query the system can run is a named, reviewed SQL statement. Adding a new query type requires an engineering review and a row in `ApprovedQueryTemplates`.
- TenantCode injection from session (not user input) means cross-state access is blocked at the query construction layer, not filtered post-query.

- Low-count suppression (<5 individuals) on query results prevents statistical re-identification of children's eligibility status — required for FERPA compliance.
- The template catalog can be extended without system changes — adding a new row in `ApprovedQueryTemplates` makes a new query type available to all states with appropriate roles.

Rejected alternatives

- ~~Arbitrary text-to-SQL (LLM generates SQL directly)~~ — cannot reliably prevent cross-state data access; SQL injection equivalent via natural language; no audit trail of what queries are possible.
- ~~Pre-built reports only (no NLP)~~ — higher support burden; state staff submit requests for data they could self-serve; slower iteration on analytics needs.

Consequences

- Users occasionally ask questions the template catalog does not cover. The rejection message should explain what types of questions are supported and offer to escalate unsupported question types for catalog addition.
- Template catalog coverage is a product decision. Starting with the most common FNS report queries (FNS-10, FNS-640, FNS-905 equivalents) and expanding from support tickets.
- `AllowedRoles` on each template controls which roles can execute which queries — financial queries require Finance role, not Caseworker.

DIS and CTS share a single unified platform from the start

Context

LINQ currently operates two separate CNP product lines: CTS (22+ states, cloud-based) and DIS (6 states: AK, HI, ME, MO/DHSS, UT, WI — legacy WinForms desktop applications with local SFTP-based data collection). These are maintained as separate codebases, separate teams, and separate deployment pipelines. DIS states use INDcFtp, UTDcFtpv2, and related WinForms tools that handle DC (Direct Certification) feed collection. Third-party proposals for CNP modernization have targeted CTS only.

Decision

The unified CNP platform serves all 28 states from a single codebase. DIS-specific DC feed collection is handled by the `IDcFeedAdapter` *interface — a thin adapter layer that abstracts SFTP, API, and file-format differences without forking the platform. DIS WinForms applications*

are retired as DIS states migrate to the new platform. DIS migration is a parallel track to CTS migration, not a Phase 4 afterthought.

Rationale

- A CTS-only platform continues to require separate DIS maintenance indefinitely — the same engineering cost that the modernization is meant to eliminate.
- The 6 DIS states represent meaningful ARR. Excluding them from the platform roadmap is a commercial decision that has not been made.
- The federal compliance requirements (FNS programs, NDL, FNS reports) are identical between CTS and DIS states. The platform's compliance engine is fully reusable.
- DC feed differences (SFTP vs. API, state-specific file formats) are integration differences, not architectural differences. The `IDcFeedAdapter` pattern contains them without forking the codebase.
- nTIG decisions in Q4 2026 apply to all CNP states. DIS states evaluating alternatives will be influenced by what they see at the October prototype — if DIS is excluded, those states have no reason to stay.

Rejected alternatives

- CTS modernization first, DIS as Phase N+1 — perpetuates two codebases; DIS states are left out of the nTIG conversation; no commercial leverage for the October prototype with DIS states.
- Separate DIS platform — doubles infrastructure, doubles compliance maintenance, doubles the federal math testing surface.

Consequences

- DIS state migration sequence must account for active WinForms deployments. The `IDcFeedAdapter` can be deployed in parallel with existing DIS tools during transition — states don't need to cut over all at once.
- MO/DHSS is a split-state case (state agency split across two departments). This requires design before migration — it is a known pending item.
- The SFSP seasonal deploy blackout (May–August) applies specifically to states running SFSP summer service. DIS states in this category (HI, ME, others) must be excluded from production deployments during their active summer service window regardless of platform track.

Workflow State Machine (named hooks) and Rules Engine have distinct, non-overlapping responsibilities

Context

Two systems in the platform evaluate conditions and route work: the Workflow State Machine (transition conditions and actions implemented as named C# hooks) and the Rules Engine (Microsoft Rules Engine with JSONB-stored rule sets evaluated against C# models). Both appear to handle "routing conditions." Without a documented boundary, a developer adding a new routing requirement has no guidance on which system to use, and the two systems risk duplicating or contradicting each other.

Decision

The Workflow State Machine governs transition eligibility and transition side effects — synchronous, transactional, within the lifecycle event. The Rules Engine governs operational consequences after a transition completes — asynchronous, post-transition, outside the transaction. These are not interchangeable. A given routing requirement belongs in exactly one system based on when it must execute and whether it can block the transition.

The boundary, precisely stated

- **State Machine (named hooks)** — answers "can this transition happen?" (condition hooks) and "what must happen atomically when it does?" (action hooks). Executes synchronously, within the database transaction. A condition hook returning false blocks the transition entirely. Examples: `NoNDLMatch`, `AllClaimErrorsResolved`, `FreezeClaimLaws`, `GeneratePYRenewalPacket`.
- **Rules Engine (Microsoft Rules Engine)** — answers "given that this transition completed, what operational work follows?" Executes asynchronously, after the transaction commits, via a Hangfire background job. Cannot block or roll back a transition. Examples: assignment routing (which reviewer receives this application), escalation timers (flag case if unreviewed for 10 days), notification triggers (send email when state changes to UnderReview).

Decision rule for developers

When adding a new routing or condition requirement, apply this test in order:

- Does it need to **prevent** a workflow transition from occurring? → Named hook (condition), registered in `WorkflowTransitionConditions`.
- Does it need to **execute as part of** a workflow transition (same transaction, must succeed for transition to complete)? → Named hook (action), registered in `WorkflowTransitionActions`.

- Does it fire **after** a transition and cannot roll it back? → Rules Engine, triggered by a Hangfire job consuming a domain event published at transition commit.
- Is it a **display or UI routing** concern (which queue does this appear in, which user sees it)? → Rules Engine or ICnpPermissionService depending on whether it's dynamic assignment or static access control.

What is explicitly out of scope for the Rules Engine

- Federal compliance math — never in JSONB rule sets, only in named hooks with `IsFederal=true`.
- Any logic that must run within a database transaction.
- Any logic that needs access to the pre-transition state (use action hooks, which receive the transition context).

Rationale

- Named hooks are compiled C# — testable, code-reviewed, type-safe. JSONB rules are data — flexible, configurable by power users, but not independently testable as first-class code artifacts.
- Putting assignment routing in the Rules Engine keeps the state machine focused on compliance logic. It also allows onshore config staff to modify escalation rules without a code deployment.
- Synchronous/transactional vs. asynchronous/post-commit is the sharpest line between the two systems. If a rule needs to be synchronous and transactional, it belongs in a named hook. There are no exceptions.

Consequences

- Every named hook must have a registration in the hook catalog (discoverable by the Workflow Override API). Every Rules Engine rule set must have a corresponding Hangfire job that consumes the domain event that triggers it.
- This boundary must be enforced in code review. A PR that puts assignment logic in a named hook, or puts a blocking condition in the Rules Engine, should be rejected.
- The Workflow Engine and Rules Engine design documents must each reference this ADR and state explicitly what belongs in the other system.

Sources

Electronic Code of Federal Regulations

Title 7 of the CFR -- Agriculture

Specific parts referenced most:

- **Part 210** (NSLP): 7 CFR Part 210 -- National School Lunch Program
- **Part 220** (SBP): 7 CFR Part 220 -- School Breakfast Program
- **Part 225** (SFSP): 7 CFR Part 225 -- Summer Food Service Program
- **Part 226** (CACFP): 7 CFR Part 226 -- Child and Adult Care Food Program

nTIG Non-Competitive Technology Innovation Grant

Non-competitive Technology Innovation Grant (nTIG) | Food and Nutrition Administration

Regulatory basis for the IsFederal guardrail

- 7 CFR Part 210 — NSLP
- 7 CFR Part 220 — SBP
- 7 CFR Part 225 — SFSP
- 7 CFR Part 226 — CACFP

FNA policy and program home

- fna.usda.gov — the renamed agency site; references to "FNS website" in the architecture should point here now
- FNA Documents & Resources — where policy memos, program guidance, and annual updates live; this is what the Policy Change Detection AI monitors
- Income Eligibility Guidelines 2026–2027 — the annual FNA update that triggers ClaimLaws changes

Security and compliance obligations

- NIST SP 800-171 — the 110 CUI security requirements referenced in the Access Governance doc; should be cited in Section 3d upstream dependencies
- FERPA (34 CFR Part 99) — applies when LINQ processes student eligibility data on behalf of school districts; relevant to the analytics guardrails and data handling policy

Federal Register (for the AI capability)

- federalregister.gov — this is the actual feed the Policy Change Detection AI monitors for regulatory updates; worth naming explicitly in Section 3c